

6

7.4.1

Suppose the heap consists of seven chunks starting at address 0. The sizes of the chunks in order are 80 30 60 50 70 20 40 bytes. When we place an object in a chunk we put it at the high end if there is enough space remaining to form a smaller chunk so that the smaller chunk can easily remain on the linked list of free space. However we cannot tolerate chunks of fewer than 8 bytes so if an object is almost as large as the selected chunk, we give it the entire chunk and place the object at the low end of the chunk. If we request space for objects of the following sized 32 64 48 16 in that order what does the free space list look like after satisfying the requests if the method of selecting chunks is

```
80, 30, 60, 50, 70, 20, 40
```

First fit:

```
32 → 48 (32) 30 60 50 70 20 40
```

```
64 → 48 (32) 30 60 50 (64 + 6) 20 40
```

```
48 → (48) (32) 30 60 50 (64 + 6) 20 40
```

```
16 → (48) (32) 14 (16) 60 50 (64 + 6) 20 40
```

Best fit:

```
32 → 80 30 60 50 70 20 8 (32)
```

```
64 → 80 30 60 50 8 (64) 20 8 (32)
```

```
48 → 80 30 60 (48 + 2) 8 (64) 20 8 (32)
```

```
16 → 80 30 60 (48 + 2) 8 (64) 8 (16) 8 (32)
```

7.5.1

What happens to the reference counts of the objects in Fig. 7.19 if

a

The pointer from *A* to *B* is deleted

✓ Answer ✓

B goes from 1 to 0.

D goes from 1 to 0.

E goes from 2 to 1.

B and *D* get dropped.

b

The pointer from X to A is deleted

✓ **Answer**

A goes from 1 to 0.

B goes from 1 to 0.

C goes from 2 to 1.

D goes from 1 to 0.

E goes from 2 to 1.

G goes from 2 to 1.

A , B , and D get dropped.

c

The node C is deleted.

✓ **Answer**

C goes from 2 to 0.

F goes from 1 to 0.

H goes from 1 to 0.

C and F get dropped.

9.1.1

a

✓ **Answer**

2 backedges,

B_2, B_3, B_5

B_3, B_4

b

✓ **Answer**

For variable a in nodes B_2, B_3, B_4, B_5

We cannot do this for b as it is used in B_6

c

✓ **Answer**

Small loop: $a + b$, e , d changes so they cannot be common subexpressions,

Large loop: $a + b$, $c - a$ along B_2 to B_5 , but not after as it is changed.

d

✓ **Answer**

e is an induction variable as it increases by 1 each loop on the small loop.

c and b are induction variables for the large loop as they increase by constant a each loop. If we do not enter the left loop, d increases by $2a$ each iteration. Similarly, if we do not enter the small loop, e also increase by a each loop. So b, c, e are a group of induction variables increasing by a each time.

e

✓ **Answer**

$d = a + b$ is always the same, it is the same as c .

Since all variables in the larger loop are induction variable, they cannot be invariants.

9.1.4

```
dp=0
i=0
L:
t2=A[i]
t4=B[i]
t5=t2*t4
dp=dp+t5
i=i+8
jmpif i<n L
```